

企业大规模AI编程落地 如何控制大模型成本？

演讲嘉宾：毛卓 技术总监 碧桂园服务

目录

CONTENT

- 01 为什么成本很重要
- 02 Token消耗在哪了
- 03 技术-优化上下文
- 04 管理-创新研发流程
- 05 人才-注重综合能力

01

为什么成本很重要

1.1 投入产出比合理的项目才有未来

投资者三问

- AI带来的收入在哪里？
- AI投入的钱算在哪里？
- AI最后能不能提高利润率？

这三件事很俗，但很重要

ROI先算清

- 传统企业做项目先算账：业务价值 vs 投入成本
- 只有算清楚了，才敢立项开工
- AI编程更需要先回答：这笔投入ROI算得过来吗？
- 投资者开始关注AI业务价值的变现路径

核心观点

- 在大规模AI编程落地之前，必须先回答：这笔投入，ROI算得过来吗？
- 只有投入产出比合理的项目才有未来，这是AI编程大规模推广的前提
- 别被“Token越来越便宜”的说法麻痹了，企业实际总支出在暴涨。Token成本下降后企业总成本仍失控？

1.2 成本与效率的关系：鱼和熊掌兼得

常见误解

- 降本和增效是矛盾的
- 要省钱就得牺牲效率，要效率就得花钱
- 这个观念在AI时代尤其危险

真实情况

- 改变敏捷研发模式，节省2倍的Token消耗
- 把研发预算减少了一半，工作效率反而翻倍
- 这不是魔法，是流程重构带来的结构性优化

正循环链条

- 节省Token降成本 → 输入更高效 → AI理解更准确 → 输出正确率更高 → 代码质量更好 → 返工率降低 → 研发效率更高
- 工具替换只能产生局部加速，无法带来系统性效能跃升
- 真正聪明的管理者重新设计整个研发流程

1.3 未来研发团队成本的计算方式

传统软件工程：平庸者的温床

- 工程师像流水线螺丝钉：接需求、写CRUD、解Bug
- 只要听话、肯加班、不出大错就是靠谱员工
- 隐性成本巨大：烂代码Bug率高，返工不断
- 1万工资的员工，实际可能消耗团队3万成本

AI时代：Token成为照妖镜

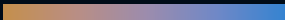
- 当AI成为标配，比拼的不是手速而是思维质量
- 平庸者：提示词模糊方案摇摆，AI反复跑偏
- 优秀者：用少量Token换来十倍产出

真实成本公式

- 优秀：月薪3万 + Token 1000元 + 提效几倍？
- 平庸：月薪1万 + Token 2万 + 提效几倍？
- **未来真实成本 = 工资 + Token消耗 + 提效几倍？**

02

Token消耗在哪了？



2.1 智能体与LLM的调用原理

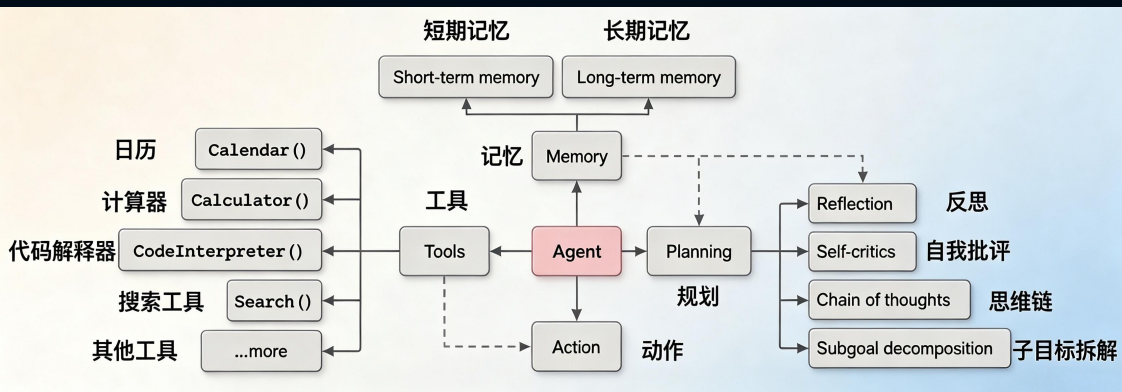
智能体的工作原理

- 智能体 (Agent) 其核心是LLM，LLM承担着大脑的角色，用于思考和规划。

Agent核心能力还包含记忆和工具：

- 记忆：用于存储短期上下文信息和长期知识信息
- 工具：承担着感官和四肢的角色

Agent基于LLM的思考和规划，通过工具获取外部的各种信息用于进一步的思考和规划，也可以通过工具执行动作对外部环境施加影响。



智能体的工作原理

《LLM Powered Autonomous Agents》中描述的Agent整体架构，包含以下核心组件：

- 规划 (Planning)**：大语言模型作为Agent的大脑负责思考和规划，而思考和规划的方式又可以分为两部分：

- 分而治之 (Task Decomposition)：对于复杂的任务，大语言模型会将其分解为多个相对简单的子任务，每个子任务包含独立的子目标，从而分而治之、逐步求解
- 自我反思 (Self-Reflection)：大语言模型会对过去的规划和执行进行自我反思，分析其中的错误，并对后续的思考 and 规划进行改进，完善最后输出的结果

- 记忆 (Memory)**：记忆是对大语言模型本身由模型结构和参数所蕴含知识的补充，记忆又可以分为短期记忆和长期记忆：

- 短期记忆，即大语言模型的上下文学习，包括提示、指示、前序步骤的大语言模型推理结果和工具执行结果等
- 长期记忆，即外部可快速检索的向量索引，这也就是目前比较流行的一种大语言模型应用的解决方案——RAG (Retrieval-Augmented Generation, 检索增强生成)

- 工具 (Tool)**：作为Agent的感官和四肢，Agent一方面可以通过工具获取外部的各种信息用于进一步的思考和规划，例如通过搜索引擎搜索某个关键词的最新信息，另一方可以通过工具执行动作对外部环境施加影响，例如调用外部系统的接口执行指令并获取执行结果

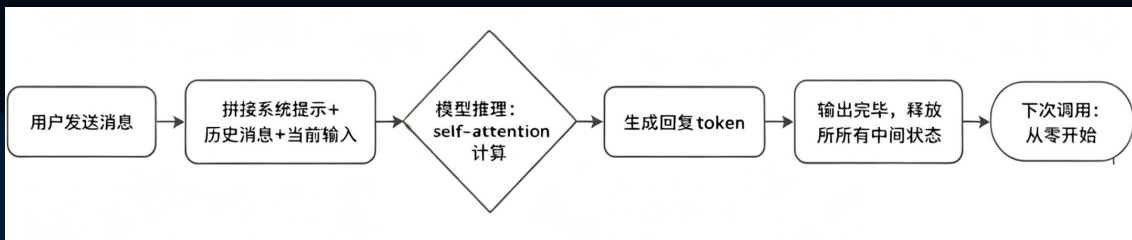
2.2 为什么大模型天生没有记忆?

Transformer算法本身没有记忆

Transformer 的自注意力机制self-attention: 根据一段输入, 计算每两个 token 之间的关联程度, 输出结果, 然后把所有中间状态全部丢弃。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

参数: Q (查询)、K (键)、V (值) 三个矩阵全部由当前输入线性变换而来, 没有任何来自上一次调用的状态参与计算。每次推理都是一个纯粹的 输入 → 输出 映射。



软件工程角度高并发的设计

- 大模型无状态设计意味着每次调用完全独立
- 同一个模型可以同时服务数万个并发用户
- 工程选择: 每次调用完全独立, 可同时服务数万并发用户
- 隐私与安全: 避免用户信息被存储和滥用

为什么用户觉得AI记住自己?

- 上下文窗口: 程序将之前对话原样重新塞回给模型
- 不是真记得: 这只是模型读到长文字
- 心理投射: 人类天生倾向将情感投射到非生命体

2.3 如何解决大模型没有记忆的问题

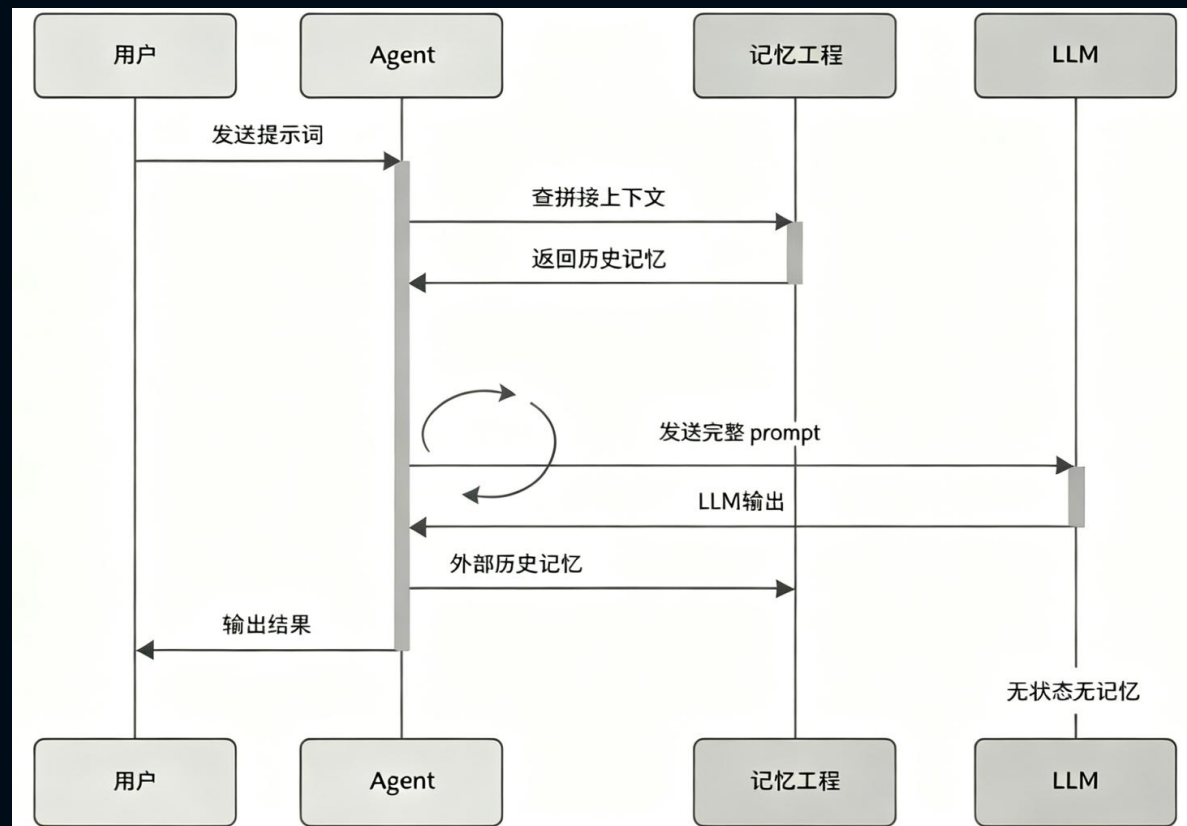
模型本身无状态，所有让Agent有记忆的方案，本质上都是在模型外部搭建一套 长期记忆系统

- 外部文件存储记忆系统：

如ChatGPT的Memory功能、Claude的CLAUDE.md文件

- Agent记忆系统（四层）：

- 工作记忆：当前的上下文窗口
- 情景记忆：记录具体发生过的事件
- 语义记忆：沉淀出的稳定事实
- 程序记忆：关于如何做事的步骤和技能



2.4 分词规则决定Token消耗量

什么是Token

- AI大模型中读取和生成文本的最小基本单元
- 理解为AI眼中的单词碎片或字母积木
- $\text{Input} + \text{Output} \leq \text{最大上下文长度}$

Token锁死大模型的三项物理极限

- ① 记忆容量（上下文窗口）
- ② 你的钱包（计费标准）
- ③ 响应速度（生成速率）

分词算法

- 不同LLM的分词器 (Tokenizer) 不同, Token的计算方式也会有差异。不同的模型使用不同的它们拆解单词的规则和词库都不一样。

同一段英文或中文, 在不同模型里消耗的Token数量可能不同。

- 中文: 在英文LLM里消耗的Token较多, 而在国产LLM中通常更省Token。主要因为复合词和成语俚语等的分词规则。
- 英文: 在中文LLM里消耗Token较多, 而在英文LLM中消耗更少, 因为中文模型会把少见的词切得更碎

一个汉字大约有多少个Token

- **简单词**: 有些词很简单, 比如“我”、“你”、“好”, 这些词都是单独一个汉字, 每个Token就是一个汉字。
- **复合词**: 有些词由两个或更多汉字组成的, 比如“喜欢”、“我的”、“女朋友”, 每个Token就大约是两个字
- **成语和短语**: 成语是由四个汉字组成, 把它们看作一个整体, 那么每个Token就大约是四个字
- **代码**: 比自然语言更费 token, 缩进的空格、换行符全都单独计数; 结构化数据走 JSON 比 YAML 更贵, 括号和引号都是要花钱的字符。

2.5 大模型输出 比 输入 更贵

物理算力消耗决定价格

大模型推理的两个阶段：

Prefill（预填充）和 Decode（解码）
直接决定了输入和输出Token单价不同

- Prefill（预填充）= 读题
- Decode（解码）= 答题
- GPU的物理成本，决定定价策略
- 输出token价格是输入的3-5倍
- 不同模型间的价差巨大

特性维度	Prefill 阶段（“读题”）	Decode 阶段（“答题”）
核心任务	一次性并行处理所有输入的Token	基于已有内容，逐字（串行）生成新的Token
计算类型	计算密集型 (Compute-Bound) 进行大量并行矩阵乘法 GPU算力 是瓶颈	内存密集型 (Memory-Bound) 频繁读取“记忆” (KV Cache) 显存带宽 是瓶颈
性能指标	TTFT (Time to First Token) 从提问到出现第一个字的时间	TPS (Tokens Per Second) 开始回答后，每秒生成Token的速度
资源消耗	矩阵乘矩阵的重运算，GPU 算力被打满，卡在算力上 (compute-bound)。算力消耗高，但持续时间短。	decode 每生成一个 token，都要把整个模型权重从显存里完整搬运一遍，运算退化成矩阵乘向量，GPU 利用率掉到两三成，大部分时间在等显存带宽 (memory-bound)。单次算力消耗低，但持续时间长，且随输出长度线性增长。
优化可能	可通过提示词缓存 (Prompt Caching) 复用计算结果，避免重复“读题”	优化空间有限，必须逐字进行，难以并行
价格对比	因为 Prefill 是高度并行的，GPU的算力可以被充分利用，处理效率极高。 因此，每个输入Token分摊的成本很低 输入价格：\$3/百万token (Sonnet 4.6)	这个过程无法被有效并行化。 每次生成一个Token，都需要将模型全部参数从显存搬运到计算单元，效率很低。 因此，每个输出Token的生成成本要高得多 输出价格：\$15/百万token (Sonnet 4.6)

2.6 上下文窗口与记忆容量

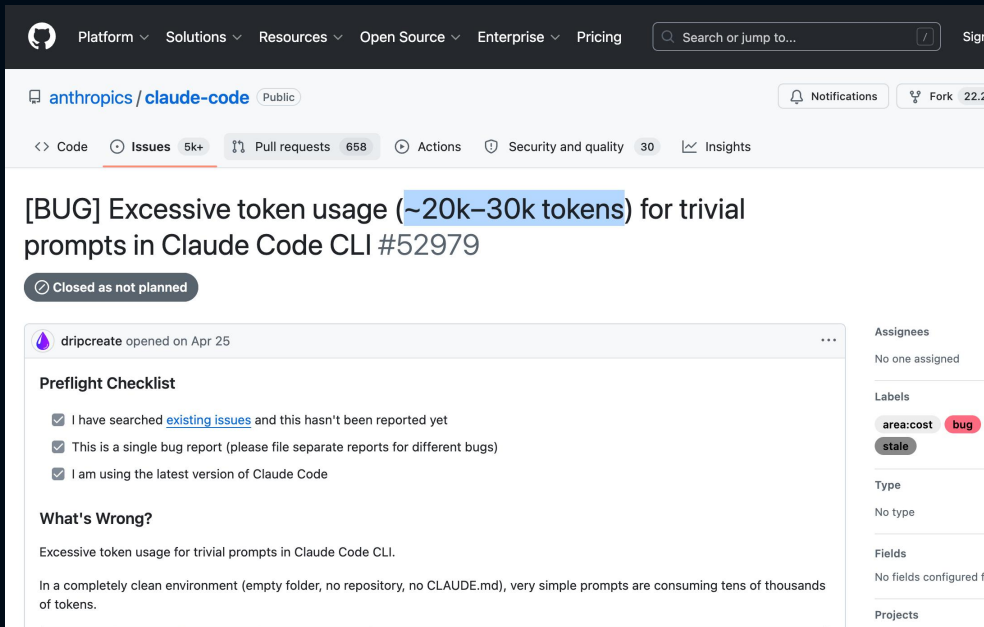
上下文窗口决定记忆

- 上下文窗口就是 AI 大模型一次能“记住”并处理的最大文字量，是AI的“短期记忆容量”
- 这个窗口大小决定了模型能读多长的文档、聊多久天而不忘事
- 窗口再大也架不住乱用

模型名称	发布	上下文窗口	最大输出	价格（输入/输出）
GLM-5.2	2026-06-15	1M	131K	输入 ¥8 / 1M tokens, 输出 ¥28 / 1M tokens
Claude Opus 4.8	2026-05-28	1M	128K	5 / 25 美元 每1M tokens
Qwen 3.7 Max	2026-05-20	1M	未明确	2.5 / 7.5 美元 每1M tokens
DeepSeek-V4 Pro	2026-04-24	1M	384K	Pro: 输入 ¥12 / 1M, 输出 ¥24 / 1M (缓存命中 ¥1 / 1M) ; Flash: 输入 ¥1 / 1M, 输出 ¥2 / 1M (缓存命中 ¥0.2 / 1M)
GPT-5.5	2026-04-23	1M	128K	5 / 30 美元 每1M tokens
Claude Opus 4.7	2026-04-16	1M	128K	5 / 25 美元 每1M tokens
Qwen3.5-Plus	2026-03-20	1M	未明确	Plus: 输入 ¥0.8 / 1M, 输出 ¥4.8 / 1M; Flash: 0.10 / 0.40 美元 每1M tokens
GPT-5.4	2026-03-05	1M	未明确	2.50 / 15.00 美元 每1M tokens
Claude Opus 4.6	2026-02-06	1M	未明确	5 / 25 美元 每1M tokens
DeepSeek V3.2	2025-12月	128K	64K	0.27 / 0.42 美元 每1M tokens
Claude Opus 4.5	2025-11-01	200K	未明确	5 / 25 美元 每1M tokens
Qwen3-Max	2025-09-24	256K	未明确	1.20 / 6.00 美元 每1M tokens
DeepSeek-V3.1	2025-08-21	128K	未明确	输入 ¥4 / 1M, 输出 ¥12 / 1M (缓存命中 ¥0.5 / 1M)
GPT-5.3 Chat	2025-08-31	128K	16K	1.75 / 14.00 美元 每1M tokens

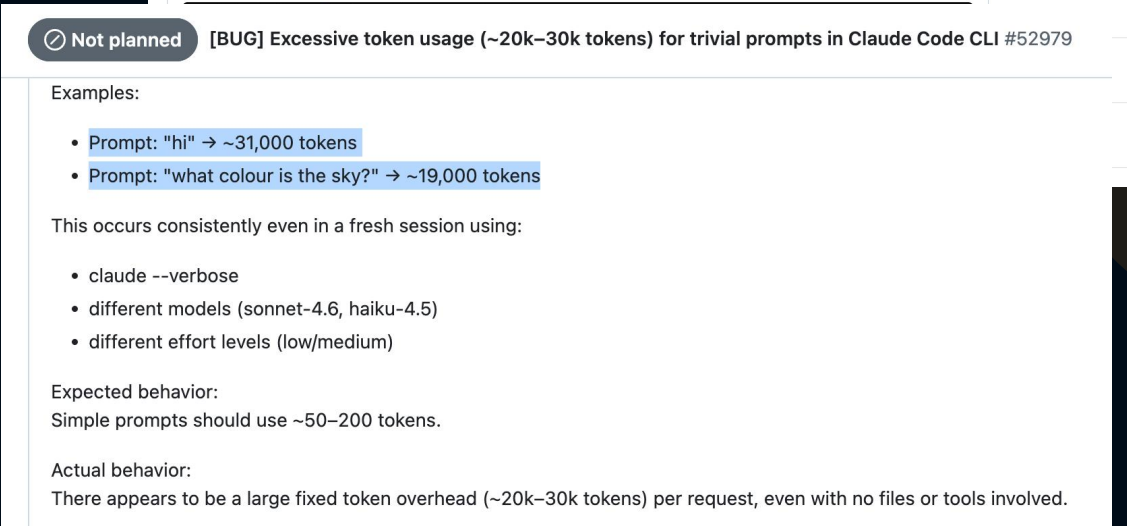
2.7 日常开发Token花在哪里了

日常编码消耗来源	Token	占比	说明
系统提示与会话初始化	~1万	10%	每次会话固定开销
工具调用累积上下文	~4万	40%	Read/Grep返回内容不断累加
AI思考与生成代码	~2万	20%	真正"干活"的部分
返工修复（幻觉/调试）	~2万	20%	AI写了错误代码，反复修改
其他（格式化/解释文字）	~1万	10%	解释性文字、不必要的格式化



Claude Code真实消耗分析

- 系统提示词
- 工具名称
- MCP工具
- Skill
- 自定义agent
- 记忆文件等



03

技术 - 优化上下文

3.1.1 大模型维度：分层路由不同级别模型

核心动作

- 不是所有任务都需要最强的模型
- 让 任务复杂度 与 模型能力 精准匹配
- 杀鸡用鸡刀，杀牛用牛刀，别什么都上最贵的
- 手动选择模型，自动选择模型

Extended Thinking按需开启

- 思考过程的Token按输出价格计费，动辄数万输出
- 简单任务别开，开了就是白烧钱
- 只有复杂推理任务才需要开启扩展思考

级别	模型	适用任务	任务难度
经济型	Qwen3.6-flash Haiku	代码补全、格式化、简单重命名、简单测试	不需要复杂推理，有明确的执行规则
均衡型	Qwen3.7-Plus Sonnet	CRUD开发、功能开发、接口实现、测试编写	需要理解上下文，但范围可控，有一定执行标准
推理型	Qwen3.7-max Glm-5.2 Opus4.8/fable	架构设计、复杂重构、疑难排查	需要深度思考和创造性思维，综合深度推理

3.1.2 大模型维度：分层路由模型 示例

Step 1: 在 ~/.claude/settings.json 中配置模型别名

```
{ "env": {  
  "ANTHROPIC_DEFAULT_HAIKU_MODEL": "qwen3.6-flash",  
  "ANTHROPIC_DEFAULT_SONNET_MODEL": "qwen3.7-plus",  
  "ANTHROPIC_DEFAULT_OPUS_MODEL": "qwen3.7-max"  
},  
  "model": "sonnet"  
}
```

Step 2: 创建子代理文件 .claude/agents/fast-worker.md

```
---model: haiku  
tools: [Read, Grep, Glob, Edit]  
description: "快速处理简单任务：格式化、typo、小重构"  
--- 你是一个快速高效的编码助手。用最少的改动处理简单任务，速度优先。
```

Step 3: 创建子代理文件 .claude/agents/deep-thinker.md

```
--- model: opus effort: high  
tools: [Read, Grep, Glob, Bash, Edit]  
description: "深度分析：复杂bug、架构设计、安全审查"  
--- 你是一个深度分析型代理。充分考虑边界情况，提供全面方案。
```

Step 4: 在 CLAUDE.md 中写路由规则

```
## 智能路由规则  
- 简单任务 (typo、格式化、简单重命名) → 委派给 fast-worker 子代理  
- 复杂任务 (架构设计、疑难bug、安全审查) → 委派给 deep-thinker 子代理  
- 日常开发 → 自己处理
```

3.2 架构维度：模块解耦 和 无状态

核心动作

- 缩小AI可触碰代码范围
- 同一个需求，不同架构差3-5倍Token
- 在Prompt层抠30%，不如在架构层省300%

对比分析

- 大泥球架构：AI每次修改需理解整个系统，Token爆炸
- 模块化低耦合：AI只需理解当前模块的上下文
- AI时代新度量：Modification Surface / Context Boundary

落地动作

- Fitness Function (ArchUnit) 把架构约束代码化
- 契约先行 (OpenAPI / Protobuf)：人定契约，AI填实现
- 模块化单体优先于伪微服务

架构形态	单任务 Token 消耗	返工率
大泥球单体	40-100 万	40%以上
分布式大泥球（伪微服务）	50-120 万	45%以上
契约化模块架构	5-15 万	低于15%
真微服务（强契约）	5-15 万	低于15%

3.3.1 输入维度：高效高质量的会话

会话长度影响上下文质量

- 对话越长越贵，一个任务一个会话
- 每个任务控制在5-8轮对话内完成
- 超过10轮时强制保存状态，开新会话
- 使用/resume续接，避免重复加载上下文

根据架构和任务原子性拆分与合并会话

- 按架构拆分会话
- 按任务原子性拆分会话
- 合并：同类小任务合并（共享上下文）

子代理独立干重活，保持主会话干净

- 大任务拆解成多个子代理，子代理的本质是“一次性探索员”。它在独立空间中执行搜索等重任务，完成后只回传关键结论并自动销毁。
- 主对话因此可免于被海量中间结果污染，始终聚焦核心逻辑，保持上下文纯净高效。
- 适合子代理的任务类型：搜索探索类、信息收集类、独立验证类

会话压缩和清理

- /compact — 有损压缩，节省>80%，可指定保留项：
示例：/compact 保留所有代码修改记录和文件路径，丢弃分析过程
- /clear — 彻底清空，完全切换任务时使用
- 铁律：一个会话只解决一个独立任务

3.3.2 输入维度：缓存命中与增量更新

缓存命中

- 不变内容只付一次费。主流模型均支持缓存命中打折，连续任务的基础上下文自动复用。

- 省Token原理：

Claude缓存命中打1折，GPT-4打5折，Gemini打2.5折

前面的内容不变，后面的任务变化，前面的内容自动命中缓存

- /resume 会复用之前的上下文，不需要重新加载CLAUDE.md 和项目文件

实施步骤

Step 1: 把不变内容放最前面

```
// CLAUDE.md 的内容 (不变) 放最前面
// 任务描述 (变化) 放最后面
# 会话开头
请读取 CLAUDE.md 了解项目规范。
# 任务描述
基于 Controller 模板，创建充电桩管理接口.....
```

Step 2: 使用 /resume 续接会话

```
# 任务做到一半，需要暂停
# 1. 让AI生成变更摘要
请生成当前任务的变更摘要，保存到 .ai-memory/session-summary.md
# 2. 关闭会话
# 3. 下次继续时:
claude --resume <session-id>
# 或者开新会话，先读取摘要:
请读取 .ai-memory/session-summary.md，继续之前的任务
```

Step 3: 任务间传递“变更摘要”而非完整上下文

```
// 任务A完成后
请生成任务A的变更摘要，包括：
- 修改了哪些文件
- 关键变更内容
- 待处理的事项
保存到 .ai-memory/task-a-summary.md

// 任务B开始时
请读取 .ai-memory/task-a-summary.md，了解任务A的变更，然后继续任务B。
```

3.3.3 输入维度：精准提示词输入

核心公式

- 公式化提问与限制行为范围
- 在什么文件里做什么，用什么方法，具体怎么干

上下文管理三原则

- 只给必要信息：每次调用只传递当前任务真正需要的上下文内容
- 排除无关文件和目录，配置 `.claudeignore`，不让AI看无用的信息
- 过滤无效信息：日志过滤、diff切片、输出摘要

错误 vs 正确示例

✗ 错误示例：

"你是一个资深Java开发者，精通Spring Boot、MyBatis等框架。请你帮我实现一个后台管理接口....."

☑ 正确示例：

"基于 Controller 模板，创建充电桩管理接口：POST /api/v1/charging-stations。字段：stationNo(String), location(String), maxPower(Double)。"

● 精简方法：

- 删掉角色声明
- 通用规范
- 礼貌用语
- 引用模板和规范文件。。。

3.3.4 输入维度：通过Skill动态加载上下文

CLAUDE.md不要装太多

- CLAUDE.md 是 Claude Code 的项目级记忆文件
- 问题：CLAUDE.md 每轮对话都会全量加载，内容过多会导致巨大的 Token 浪费
- 瘦身 CLAUDE.md —— 只保留最常用、最核心的规则（如技术栈、关键目录、禁用技术等）

低频指令迁移到 Skills

- 按低频使用的规范（如代码审查、部署流程）做成Skill
- 按需加载的 Skill，只有触发时才消耗 Token

原则：高频规则用CLAUDE.md，低频指令用Skill

3.3.5 输入维度：MCP按需加载

MCP隐藏式消耗Token

- MCP的工具定义每次对话都会全量加载到系统提示词中
- 即便不使用MCP，也会占用大量 Token

按需动态加载

- 按需开关：只开启当前任务真正需要的 MCP，用完即关
- 启用 MCP-CLI 模式：如果版本支持，开启该功能可将 MCP 相关 Token 消耗降低约 85%，实现工具定义按需获取。
- 定期检查：使用 /context 命令查看 MCP 工具占用的 Token，做到心中有数。

3.4.1 输出维度：精简输出内容

精简输出3指令

AI默认会输出大量解释性文字。

一句指令去掉多余输出Token。

三句金指令：

- ① 只输出代码
- ② 不要重复已有代码
- ③ 用diff格式输出

让AI按格式输出，并基于模板只填差异部分。

实施步骤

Step 1: 在 CLAUDE.md 中写明输出规范

输出规范

- 不要重复已有代码，只输出修改部分
- 只输出代码，不要解释
- 用diff格式输出变更 - 不要输出“好的”“当然可以”等确认语
- 不要输出“总结一下”“注意事项”等结尾语

Step 2: 在每次Prompt中强化输出要求

// 在每个Prompt的结尾加上：只输出代码，不要解释
// 或者更严格：只输出修改后的代码，不要重复未修改的部分，不要解释
// 对于修改任务：用diff格式输出变更，只输出修改的行

Step 3: 使用 JSON Schema 约束输出格式

// 让AI按固定格式输出，避免自由发挥 请按以下JSON格式输出代码审查结果：

```
{ "issues":  
  [  
    {  
      "file": "文件路径",  
      "line": 行号,  
      "severity": "error/warning/info",  
      "description": "问题描述",  
      "suggestion": "修改建议"  
    }  
  ]  
}
```


3.4.2 输出维度：结构化输出 + 模板复用

创建结构化模板

让AI按格式输出，并基于模板只填差异部分。

建立代码模板库 (.ai-memory/templates/)

- *Controller.template.java* — 标准RESTful接口模板
- *Service.template.java* — 标准业务逻辑模板
- *Mapper.template.java* — MyBatis-Plus Mapper模板
- *CreateReq.template.java* — 创建请求DTO模板
- *VO.template.java* — 响应VO模板
- *PageReq.template.java* — 分页请求DTO模板
- *IntegrationTest.template.java* — 集成测试模板

实施步骤

Step 1: 在 CLAUDE.md 中引用模板

代码模板

创建新类时，先读取 .ai-memory/templates/ 下对应的模板：

- Controller → *Controller.template.java*
- Service → *Service.template.java*
- Mapper → *Mapper.template.java*
- 集成测试 → *IntegrationTest.template.java*

基于模板生成代码，只替换占位符 (*{Entity}*、*{entity}*、*{api}*等)

Step 2: 在Prompt中引用模板

// 不要这样说：

请实现一个管理系统后台的Controller，包含
create/getByld/page/update/delete方法.....

// 应该这样说：

基于 *Controller.template.java* 模板，创建管理系统后台接口：

- Entity: *ChargingStation*
- entity: *chargingStation*
- api: *charging-stations*
- 需要的方法: *create, getByld, page, update, delete*

3.5 软件工程维度：Harness工程

为什么Harness能省Token

● 核心思想：

与其让AI每次都从零写代码，不如提前搭好“脚手架”，让AI只需要填充差异部分。

没有 Harness	有 Harness	效果
AI从零生成整个文件	AI基于模板填充差异	省 60-80%
AI猜测项目结构和技术栈	结构已定义，AI只需遵守	减少幻觉和返工
AI写代码后需要大量调试	测试框架自动验证快速反馈	减少调试循环

层	名称	具体内容	详细介绍
1	项目脚手架	项目结构、模块划分、技术栈定义	清晰的Harness工程目录
2	代码模板库	Controller/Service/Mapper等标准模板	AI从“写代码”变成“填代码”，只需要替换占位符
3	API契约	接口定义、请求/响应Schema	用 OpenAPI 规范文件明确定义所有接口，AI直接读取，不用猜测。减少“AI猜错→修改→再猜错”的循环。
4	测试	单元测试框架、Mock配置、测试数据	- JUnit5 + Mockito + SpringBoot Test 框架预置 - BaseIntegrationTest 基类 (H2内存库 + Mock外部依赖) - TestDataLoader 自动加载测试数据 - 测试模板 (AI只填测试数据和业务断言) 测试框架自动告诉AI哪里写错了，比AI自己“猜”bug准确得多。减少调试循环约 50-70%
5	构建	Dockerfile、Maven配置、环境配置	- 统一 Maven 父 POM，锁定所有依赖版本 - Docker 多阶段构建，编译环境和运行环境隔离 - Checkstyle — Java 代码规范自动检查 - SpotBugs — 静态Bug检测 (空指针、资源泄漏) - 自动化测试 — 每次提交自动运行单元测试和集成测试
6	CI/CD	自动化流水线、代码检查、部署脚本	CI流水线自动运行，有问题精准反馈给AI，AI根据真实错误修复，而不是靠自己“猜”。格式类问题在提交前自动修复，不浪费Token。

04

管理-创新研发流程

4.1.1 AI编程时代 的新研发流程

打破团队边界，打破职位边界，打破传统管理模式，不破不立。人尽其用，发挥所长，强者带教，重塑研发流程！



4.1.2 研发流程重构：多人写PRD+少数人写代码

核心逻辑

- 把成本花在执行AI一次做对上
- 而不是花在让AI反复做错再改上
- 新模式：所有人一起写PRD → 一起审核校验
- 确保PRD准确 → 最后少数人写代码
- **结果：研发效率翻了一倍，Token消耗减少一半**

流程对比

- 传统：少数人写需求 → 多数人写代码
- AI时代：多人写需求 → 少数人写代码
- 多人验证需求准确性
- 让AI在编写前就理解清楚需求

环节	AI+流程重构后	提效逻辑
需求沟通与对齐	多人协同写PRD，前置消化歧义	减少开发阶段的来回确认，节省约50%的沟通成本
AI代码生成	PRD精准，AI一次生成准确率高	精准PRD让AI返工率从40%降至15%以下
调试与返工	需求清晰，边界条件已在PRD中覆盖	返工量大幅减少，节省约60%的调试时间
Code Review	代码结构清晰，Review聚焦核心逻辑	Review周期缩短约40%

4.2 研发流程瓶颈：人工代码评审慢

AI时代的Review挑战

- AI让写代码快了2-3倍，但Review只快了30%
- 每人每天代码量：200行 → 600行
- Reviewer负担从检查作者逻辑变成验证AI意图
- 微PR累积风险：整体架构在悄悄腐化

AI代码5个新增检查维度

- 幻觉API：AI可能编造不存在的API
- License/IP污染
- 安全漏洞
- 架构约束符合度
- 可维护性

四层防御 Defense in Depth

- L1 自动化检查：静态分析、安全扫描
- L2 AI辅助Review：验证意图、架构符合度
- L3 人工Review：业务逻辑、关键判断
- L4 测试验证：覆盖率、断言质量

4.3 防返工：质量、测试、可维护性

AI编程的效能度量

- 效率：代码接受率、PR周期
- 质量：引入Bug率、覆盖率
- 成本：单需求开发成本
- 团队：采纳率、满意度

测试结果的虚假安全

- 覆盖率高，但关键路径断言低
- AI生成测试和代码同源错误：

AI 写代码用了错误假设，AI 写测试用了同样的错误假设 → 测试和代码一起错

生成代码的维护难度

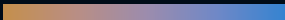
- 代码读者从人变成人与AI
- 沿用旧规范 = 制造隐形技术债
- 注释：简洁 → 结构化
- 命名：简短 → 明确无歧义

代码库的新角色：AI的操作系统

- 代码规范需要面向AI重构
- 文档体系升级：ADR、CLAUDE.md、Decision Log
- 模块边界从软约定变为强契约（OpenAPI）

05

人才 - 注重综合能力



5.1 对Vibe Coding和规约编程的清醒认知



AGENTIC AI
超级智能系统架构峰会

什么是Vibe Coding

- 仅凭感觉或模糊想法让AI生成代码
- 完全不理解背后的逻辑和风险
- 创业小公司系统挂了就挂了
- 但大型企业发生故障时，掌控者有能力修复吗？

Harness 和 规约编程

- 以人为本，人始终是软件产品的主人
- 技术够强才能SDD 和 Harness，才能掌控AI
- 驾驭才能避免Vibe Coding的黑盒
- 人才能给老板兜底

能力差距在AI时代被Token放大

- 同样的任务，不同的人，Token消耗可以差出整整一倍
- 优秀工程师用少量Token换来数十倍产出
- 招工资高但能力强的工程师，企业支出综合成本更低
- 招工资低但能力差，企业支出综合成本更高

5.2 工程师能力决定Token消耗

优秀工程师

- **架构能力** → AI产出系统稳定，减少重构成本
- **逻辑清晰** → 一次指令到位，Token消耗低
- **沟通精湛** → 与AI和人协作顺畅，减少返工
- **工程理解深** → 全流程提效
- **业务思维强** → 做有价值的事

能力不足者

- 提示词模糊、技术方案摇摆
- 反复沟通确认，大量Token消耗在对齐认知上
- Token集中在对齐认知而非产出价值
- 同样的AI工具，Token消耗可以差出一个数量级

用流程制度倒逼Token消耗优化

- 通过机制、制度来保障项目预算不超标，员工主动降低Token
- 不只看“省了多少Token”，更要看“每个Token产出了多少业务价值”
- 一个成功的组织不一定是AI花钱最多的，而是那些能衡量成果、做出明智决策，并将投资导向业务目标的组织

THANKS

演讲嘉宾：毛卓
